COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S2
DATE PERFORMED:	02/10/26	**DATE SUBMITTED:**	02/10/26
NAME:	Cruz, Elijah J	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 10 GRAPHS

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Create C++ code for graph implementation utilizing adjacency matrix and adjacency list

CODE

main.cpp > graphEdge > start_ver

```cpp
#include <iostream>

// stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};

// structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};

class DiaGraph {
    int N; // number of nodes in the graph

    // insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;
        newNode->next = head; // point new node to current head
        return newNode;
    }

public:
    adjNode** head; // adjacency list as array of pointers

    // Constructor
    DiaGraph(graphEdge edges[], int n, int N) {
        this->N = N;
        // allocate and zero-initialize head pointers for all vertices
        head = new adjNode*[N]();

        // construct directed graph by adding edges to it
        for (int i = 0; i < n; i++) {
            int start_ver = edges[i].start_ver;
            int end_ver = edges[i].end_ver;
            int weight = edges[i].weight;

            // insert at the beginning
            head[start_ver] = getAdjListNode(end_ver, weight, head[start_ver]);
        }
    }

    // Destructor
    ~DiaGraph() {
        for (int i = 0; i < N; i++) {
```

OUTPUT

```
PS C:\Users\TIPQC\Documents\cruz final> c
Graph adjacency list
(start_vertex, end_vertex, weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)

PS C:\Users\TIPQC\Documents\cruz final>
```

main.cpp

main.cpp > graphEdge > start_ver

```
14  class DiaGraph {
26  public:
47      ~DiaGraph() {
48          for (int i = 0; i < N; i++) {
55          }
56          delete[] head;
57      }
58
59      // non-copyable (class owns raw memory)
60      DiaGraph(const DiaGraph&) = delete;
61      DiaGraph& operator=(const DiaGraph&) = delete;
62  };
63
64  // print all adjacent vertices of given vertex
65  void display_AdjList(adjNode* ptr, int i) {
66      while (ptr != nullptr) {
67          std::cout << "(" << i << ", " << ptr->val
68                    << ", " << ptr->cost << ") ";
69          ptr = ptr->next;
70      }
71      std::cout << std::endl;
72  }
73
74  // graph implementation
75  int main() {
76      // (x, y, w) -> edge from x to y with weight w
77      graphEdge edges[] = {
78          {0, 1, 2}, {0, 2, 4}, {1, 4, 3}, {2, 3, 2}, {3, 1, 4}, {4, 3, 3}
79      };
80
81      int N = 6; // number of vertices in the graph
82
83      // calculate number of edges
84      int n = sizeof(edges) / sizeof(edges[0]);
85
86      // construct graph
87      DiaGraph diagraph(edges, n, N);
88
89      // print adjacency list representation of graph
90      std::cout << "Graph adjacency list " << std::endl
91                << "(start_vertex, end_vertex, weight):" << std::endl;
92
93      for (int i = 0; i < N; i++) {
94          // display adjacent vertices of vertex i
95          display_AdjList(diagraph.head[i], i);
96      }
97
98      return 0;
```

```cpp
#include <iostream>

// structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};

class DiaGraph {
    int N;  // number of nodes in the graph
public:
    int matrix[10][10];     // adjacency matrix
    // Constructor
    DiaGraph(graphEdge edges[], int n, int N) {
        this->N = N;
        // initialize all cells to 0
        for (int i = 0; i < N; i++)
            for (int j = 0; j < N; j++)
                matrix[i][j] = 0;
        // construct directed graph by adding edges to it
        for (int i = 0; i < n; i++) {
            int start_ver = edges[i].start_ver;
            int end_ver = edges[i].end_ver;
            int weight = edges[i].weight;
            matrix[start_ver][end_ver] = weight;
        }
    }
};

// print the adjacency matrix
void display_AdjMatrix(DiaGraph& g, int N)
{
    std::cout << "    ";
    for (int i = 0; i < N; i++)
        std::cout << i << " ";
    std::cout << std::endl;

    for (int i = 0; i < N; i++) {
        std::cout << i << " | ";
        for (int j = 0; j < N; j++)
            std::cout << g.matrix[i][j] << " ";
        std::cout << std::endl;
    }
}
```

```cpp
// graph implementation
int main()
{
    // graph edges array.
    graphEdge edges[] = {
        // (x, y, w) -> edge from x to y with weight w
        {0,1,2},{0,2,4},{1,4,3},{2,3,2},{3,1,4},{4,3,3}
    };
    int N = 6;      // Number of vertices in the graph
    // calculate number of edges
    int n = sizeof(edges)/sizeof(edges[0]);
    // construct graph
    DiaGraph diagraph(edges, n, N);
    // print adjacency matrix representation of graph
    std::cout << "Graph adjacency matrix" << std::endl;
    display_AdjMatrix(diagraph, N);
    return 0;
}
```

Run ▶ Output

```
Graph adjacency matrix
    0 1 2 3 4 5
0 | 0 2 4 0 0 0
1 | 0 0 0 0 3 0
2 | 0 0 0 2 0 0
3 | 0 4 0 0 0 0
4 | 0 0 0 3 0 0
5 | 0 0 0 0 0 0
```

Observation/Analysis:

The adjacency list stores each vertex's outgoing edges as a linked list, so the graph is directed: the edge {0,1,2} appears only in vertex 0's list, and the cycle shows that edges can loop back. New nodes are then inserted at

the head of each list, so vertex 0 prints (0, 2, 4) before (0, 1, 2), the reverse of the order the edges were given. Vertex 5 prints an empty line because no edge starts there, though it is still a valid vertex.

ILO B: Create C++ code for implementing graph traversal algorithms such as Breadth-First and Depth-First Search

CODE	OUTPUT

B.1. Depth-First Search

```
template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};

    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});

    return G;
}

template <typename T>
void test_DFS()
{
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    // Run DFS starting from vertex ID 1 and print the order
    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order)
        std::cout << v << std::endl;
}

int main()
{
    using T = unsigned;
    test_DFS<T>();
    return 0;
}
```

```
PS C:\Users\TIPQC\Documents\cruz final> cd "c:\Users\TIPQC\Documents\cruz final\"
$?) { .\main2 }
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
PS C:\Users\TIPQC\Documents\cruz final>
```

B.2. Breadth-First Search

```cpp
template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest)
{
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    queue.push(1); // Assume that BFS always starts from vertex ID 1
    while (!queue.empty())
    {
        auto current_vertex = queue.front();
        queue.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
                queue.push(e.dest);
        }
    }
    return visit_order;
}

template <typename T>
void test_BFS()
{
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    // Run BFS starting from vertex ID 1 and print the order
    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);
    for (auto v : bfs_visit_order)
        std::cout << v << std::endl;
}
```

```
 if ($?) { g++ main2.2.cpp -o main2.2 } ; if ($?) { .\main2.2 }
PS C:\Users\TIPQC\Documents\cruz final> cd "c:\Users\TIPQC\Documents\cruz final\"
if ($?) { .\main2.2 }
PS C:\Users\TIPQC\Documents\cruz final> cd "c:\Users\TIPQC\Documents\cruz final\"
if ($?) { .\main2_2 }
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},

BFS Order of vertices:
1
2
5
4
8
3
6
7
PS C:\Users\TIPQC\Documents\cruz final>
```

Observation/Analysis:

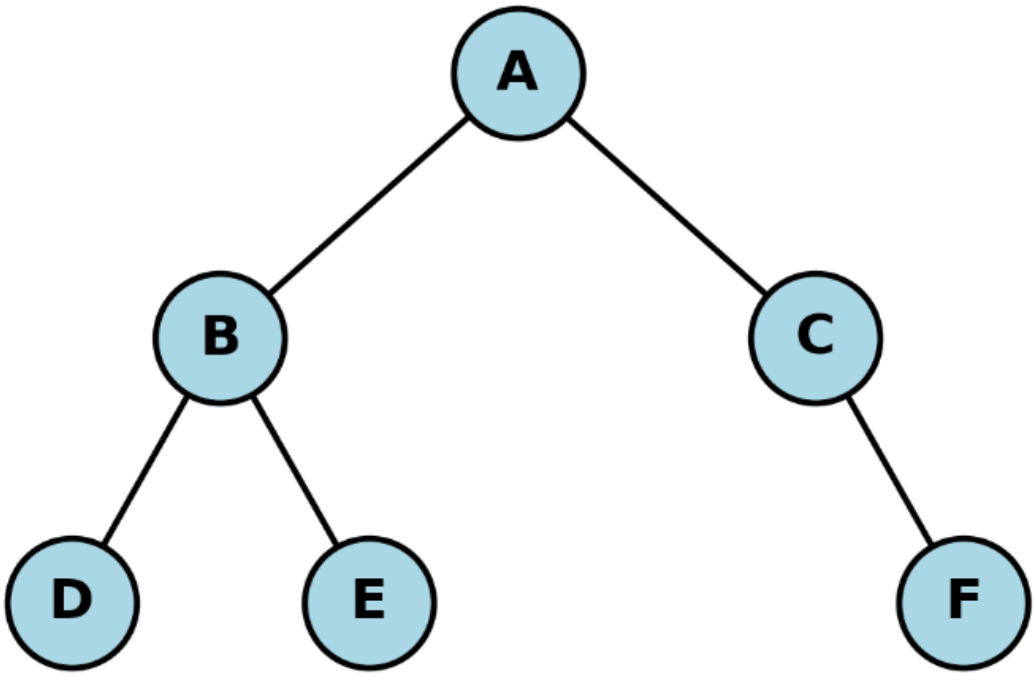
DFS goes deep first. Because the stack is last-in-first-out, it pops the last neighbor pushed (5 before 2) and follows that path as far as it can before backtracking. BFS visits vertex 1's neighbors (2, 5), then their neighbors (4, 8), then the rest (3, 6, 7), so it expands level by level. Both visit all 8 vertices exactly once, but in different orders.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

1. **A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore another vertex from the same vertex. Discuss which algorithm would be most helpful to accomplish this task.**
   DFS or Depth-First Search, as the traveler follows one path as deep as possible, backtracks when he hits a dead end, and then explores the other unvisited neighbors from the vertex he just backtracked to. Which is exactly DFS's go deep then backtrack behavior, implemented with a stack.

2. **Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.**



Traversal (DFS)	Strategy	Visit Order	Rule
Preorder	Depth	A B D E C F	Node, Left, Right
Inorder	Depth	D B E A C F	Left, Node, Right
Postorder	Depth	D E B F C A	Left, Right, Node

Level order	Breadth	A B C D E F	Level by level

PSEUDOCODE

```
PREORDER(node)
   if node is empty, return
   visit node
   PREORDER(node.left)
   PREORDER(node.right)

INORDER(node)
   if node is empty, return
   INORDER(node.left)
   visit node
   INORDER(node.right)

POSTORDER(node)
   if node is empty, return
   POSTORDER(node.left)
   POSTORDER(node.right)
   visit node

LEVEL-ORDER(root)
   create an empty queue and add root
   while queue is not empty
      node = remove first item from queue
      visit node
      if node.left exists, add it to queue
      if node.right exists, add it to queue
```

CODE

```cpp
#include <iostream>
#include <queue>
using namespace std;

struct Node {
    char data;
    Node* left;
    Node* right;

    Node(char value) {
        data = value;
        left = NULL;
        right = NULL;
    }
};

void preorder(Node* node) {
    if (node == NULL)
        return;
    cout << node->data << " ";
    preorder(node->left);
    preorder(node->right);
}

void inorder(Node* node) {
    if (node == NULL)
        return;
    inorder(node->left);
    cout << node->data << " ";
    inorder(node->right);
}

void postorder(Node* node) {
    if (node == NULL)
        return;
    postorder(node->left);
    postorder(node->right);
    cout << node->data << " ";
}

void levelOrder(Node* root) {
    queue<Node*> q;
    q.push(root);

    while (!q.empty()) {
        Node* node = q.front();
        q.pop();
        cout << node->data << " ";

        if (node->left != NULL)
            q.push(node->left);
        if (node->right != NULL)
            q.push(node->right);
    }
}
```

```cpp
int main() {
    // build the sample tree
    Node* root = new Node('A');
    root->left = new Node('B');
    root->right = new Node('C');
    root->left->left = new Node('D');
    root->left->right = new Node('E');
    root->right->right = new Node('F');

    cout << "Preorder:    ";
    preorder(root);

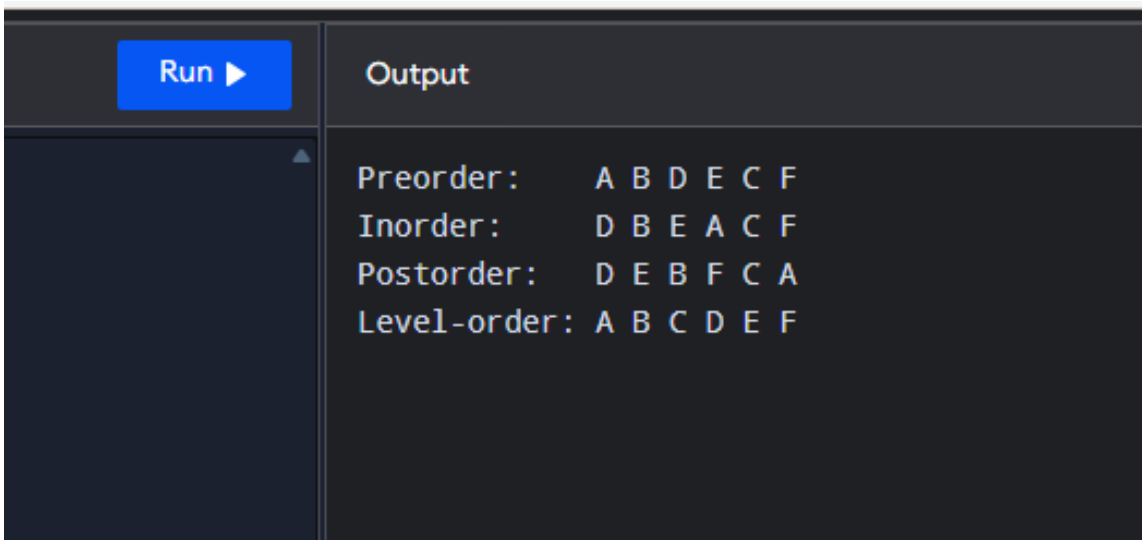
    cout << "\nInorder:     ";
    inorder(root);

    cout << "\nPostorder:   ";
    postorder(root);

    cout << "\nLevel-order: ";
    levelOrder(root);
    cout << endl;

    return 0;
}
```

OUTPUT



ANALYSIS

The equivalent of DFS in trees is depth-first traversal, which comes in three forms: preorder, inorder, and postorder. They all go as deep as possible down one branch before backing up, and they differ only in when the current node is visited. Therefore, the tree equivalent of BFS is level-order traversal, which visits the tree one level at a time. In addition, the DFS versions dive down the left side first (A, B, D) and only then come back up to reach E and C. The BFS finishes then the whole row B, C before it touches D, E, F. DFS uses a stack (the recursion call stack does this for you), while BFS uses a queue.

As for the code implementation, all four traversals would visit every node at once. The only difference is memory: DFS needs space proportional to the tree's height, while BFS needs space proportional to the widest level.

3. **In the performed code, what data structure is used to implement the Breadth First Search?**
   A queue, which is first-in-first-out and gives the level-by-level order. The code also uses the std::set to make sure we track visited vertices.

4. **How many times can a node be visited in the BFS?**
   Only once. Each vertex is processed (marked visited and added to the visit order) exactly one time as taught in our recent lecture. In the given code a vertex can be enqueued more than once, since neighbors are pushed without checking visited, but the visited check discards the duplicates. And Vertex 1 is pushed three times in the BFS trace, for example, but is only visited once.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Gemini

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: *explanation of concepts and code*

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: Summarization of concepts and how to implement BFS and DFS

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.